MyCodeSchool

React + TypeScript — teorie

10 noțiuni · fiecare are un demo în `src/demos/`

1. Componentă și JSX

O componentă React este o **funcție** care întoarce ce se vede pe ecran. Numele începe cu literă mare, ca React să o deosebească de tag-urile HTML.

Ce întoarce arată ca HTML, dar e **JSX**: HTML scris în fișierul TypeScript (`.tsx`). Componenta întoarce un singur element părinte; dacă nu vrei un `<div>` în plus, folosești fragmentul `<>...</>`.

```
export default function Profile() {
  return (
    <>
      <h1>Ion Popescu</h1>
      <p className="hint">Programator</p>
    </>
  );
}
```

Ca în Java: componenta e ca o clasă care are o singură metodă, `render()`; aici e doar funcția.

> **Capcană:** în JSX se scrie `className`, nu `class` (care e cuvânt rezervat în TypeScript). Fiecare tag se închide: `<br />`, `<input />`.

2. Variabile și funcții cu tip

În TypeScript, după numele variabilei scrii tipul: `string` (text), `number` (orice număr, întreg sau zecimal), `boolean` (`true` / `false`). `const` nu se mai poate schimba, `let` se poate.

La funcții pui tip pe fiecare parametru și pe ce întoarce. În JSX, orice expresie stă între acolade: `{name}`, `{age + 1}`, `{greet(name)}`.

```
const name: string = "Ana";
const age: number = 20;

function greet(person: string): string {
  return "Salut, " + person + "!";
}

return <h1>{greet(name)} Ai {age} ani.</h1>;
```

Ca în Java: `String name = "Ana";` devine `const name: string = "Ana";`; tipul vine după nume, nu înainte. `int` și `double` sunt amândouă `number`.

> **Capcană:** un `boolean` pus direct în JSX nu apare pe ecran; îl afișezi cu `{isStudent.toString()}`.

3. Evenimente: onClick

Legi o funcție de un buton prin `onClick`. Dai **numele** funcției, fără paranteze: React o cheamă la click.

Dacă funcția are nevoie de un argument, o înveleși într-o funcție săgeată: `onClick={() => choose("Roșu")}`.

O variabilă simplă (`let`) se schimbă la click, dar **ecranul nu se redesenează**: React nu știe că s-a schimbat ceva. Vezi valoarea doar în consolă (F12).

```
let x: number = 0;

function increment(): void {
  x = x + 1;
  console.log(x);
}

return <button onClick={increment}>Apasă {x}</button>;
```

Ca în Java: `void` înseamnă, ca în Java, că funcția nu întoarce nimic. `console.log` e `System.out.println`.

> **Capcană:** `onClick={increment()}` cheamă funcția imediat, la desenare, nu la click.

4. Starea: useState

Ca ecranul să se schimbe, valoarea trebuie ținută în **stare**. `useState` întoarce două lucruri: valoarea curentă și funcția care o schimbă.

Când chemi `setX(...)`, React redesenează componenta cu noua valoare. Tipul îl scrii între `< >`.

```
import { useState } from "react";

const [x, setX] = useState<number>(0);

function increment(): void {
  setX(x + 1);
}
```

Ca în Java: e ca un câmp privat cu setter, numai că setter-ul anunță și ecranul.

> **Capcană:** nu modifici starea direct (`x = x + 1`): mereu prin `setX`. `useState` se cheamă doar sus în componentă, nu în `if` sau în `for`.

5. Input controlat

Un input „controlat” își ia valoarea din stare (`value`) și, la fiecare tastă, scrie în stare (`onChange`). Textul tastat e mereu în variabilă, gata de folosit.

Valoarea dintr-un input e mereu **text**. Pentru numere o transformi cu `Number(...)`.

```
const [name, setName] = useState<string>("");

<input value={name} onChange={(e) => setName(e.target.value)} />

const [age, setAge] = useState<number>(0);
<input type="number" value={age}
  onChange={(e) => setAge(Number(e.target.value))} />
```

> **Capcană:** fără `onChange`, input-ul cu `value` rămâne blocat: nu poți scrie în el.

6. Condiții în JSX: ? : și &&

În JSX nu poți scrie `if`. Ai două forme scurte: `condiție ? A : B` alege între două variante, iar `condiție && A` afișează A doar când condiția e adevărată.

```
{age >= 18 ? <p>Adult</p> : <p>Minor</p>}

{age > 100 && <p className="red">Sigur?</p>}
```

Ca în Java: operatorul ternar `? :` e identic cu cel din Java.

> **Capcană:** cu `&&`, dacă în stânga ai un număr 0, pe ecran apare „0”. Folosește o comparație: `list.length > 0 && ...`.

7. Liste: type, T[] și map

Un obiect cu mai multe câmpuri primește un tip cu `type`. O listă de astfel de obiecte are tipul `User[]`.

Ca să afișezi o listă, o transformi cu `map` într-o listă de elemente JSX. Fiecare element primește un `key` unic, de obicei `id`-ul.

```
type User = { id: number; name: string; age: number };

const users: User[] = [
  { id: 1, name: "Ana", age: 20 },
  { id: 2, name: "Radu", age: 17 },
];

<ul>
  {users.map((user) => <li key={user.id}>{user.name}</li>)}
</ul>
```

Ca în Java: `type User` e ca o clasă doar cu câmpuri (un record); `User[]` e `List<User>`.

> **Capcană:** fără `key` lista merge, dar React dă avertisment în consolă și poate încurca elementele când lista se schimbă.

8. if / else

`if / else if / else` se scrie exact ca în Java. În React îl pui **într-o funcție** care întoarce rezultatul, iar în JSX chemi funcția.

Condițiile se citesc de sus în jos și se oprește la prima adevărată, deci ordinea contează.

```
function category(age: number): string {
  if (age < 14) {
    return "Copil";
  } else if (age < 18) {
    return "Adolescent";
  } else {
    return "Adult";
  }
}

return <p>{category(age)}</p>;
```

Ca în Java: identic. Comparația se face cu `===` și `!==` (trei semne), nu cu `==`. Pentru text e tot `===`, nu `.equals()`.

9. for și for…of

`for` clasic e la fel ca în Java (doar că `let i` în loc de `int i`). `for (const x of list)` parcurge elementele unei liste, ca `for (String x : list)`.

Bucla se scrie înainte de `return`, sau într-o funcție; în JSX afișezi doar rezultatul. Ca să construiești o listă pornești de la `[]` și adaugi cu `push`.

```
let sum: number = 0;
for (let i = 1; i <= 5; i++) {
  sum = sum + i;
}

const squares: number[] = [];
for (const n of [1, 2, 3]) {
  squares.push(n * n);
}
```

Ca în Java: `squares.push(x)` e `list.add(x)`; `squares.length` e `list.size()`.

> **Capcană:** nu poți scrie `for` direct între acoladele din JSX; acolo folosești `map`.

10. Metode de array: map, filter, find, reduce

Listele au metode care primesc o funcție săgeată și fac o buclă pentru tine:

```
const numbers: number[] = [1, 2, 3, 4, 5, 6];

numbers.map((n) => n * 2);          // [2, 4, 6, 8, 10, 12]
numbers.filter((n) => n % 2 === 0); // [2, 4, 6]
numbers.find((n) => n > 3);         // 4 (sau undefined)
numbers.reduce((acc, n) => acc + n, 0); // 21
numbers.includes(5);                // true
numbers.join(", ");                 // "1, 2, 3, 4, 5, 6"
```

Ca în Java: sunt `stream().map`, `filter`, `findFirst`, `reduce` din Java, fără `stream()` și fără `collect`: întorc direct o listă nouă.

> **Capcană:** `find` întoarce `undefined` când nu găsește nimic; de aceea tipul e `Product | undefined` și verifici cu `? :` înainte să folosești rezultatul.